

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Applied Business Analytics

Roll No.: T013	Name: Saqlain Khan
Class: TYIT	Batch: 1
Date of Assignment: 24-07-2026	Date/Time of Submission: 24-07-2026

PRACTICAL 1

AIM :- Introduction to Business Analytics

A) Collect data from a real-life business scenario and perform exploratory data analysis (EDA) to gain insights into the dataset.

- Import the Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

✓ 0.5s

- Load the Dataset

```
#Load dataset
df = pd.read_csv("games.csv")
```

✓ 0.0s

- Display the First 5 Rows

```
#Display First 5 Records
df.head()
```

✓ 0.0s

	Rank	Name	Platform	Year	Genre	Publisher	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sales	Re
0	1	Wii Sports	Wii	2006.0	Sports	Nintendo	41.49	29.02	3.77	8.46	82.74	J
1	2	Super Mario Bros.	NES	1985.0	Platform	Nintendo	29.08	3.58	6.81	0.77	40.24	J
2	3	Mario Kart Wii	Wii	2008.0	Racing	Nintendo	15.85	12.88	3.79	3.31	35.82	J
3	4	Wii Sports Resort	Wii	2009.0	Sports	Nintendo	15.75	11.01	3.28	2.96	33.00	J
4	5	Pokemon Red/Pokemon Blue	GB	1996.0	Role-Playing	Nintendo	11.27	8.89	10.22	1.00	31.37	J

- Display the Last 5 Rows

```
#Display Last 5 Records
df.tail()
```

✓ 0.0s Python

	Rank	Name	Platform	Year	Genre	Publisher	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sale
	16593	Woody Woodpecker in Crazy Castle 5	GBA	2002.0	Platform	Kemco	0.01	0.00	0.0	0.0	0.0
	16594	Men in Black II: Alien Escape	GC	2003.0	Shooter	Infogrames	0.01	0.00	0.0	0.0	0.0
	16595	SCORE International Baja 1000: The Official Game	PS2	2008.0	Racing	Activision	0.00	0.00	0.0	0.0	0.0
	16596	Know How 2	DS	2010.0	Puzzle	7G//AMES	0.00	0.01	0.0	0.0	0.0
	16597	Spirits & Spells	GBA	2003.0	Platform	Wanadoo	0.01	0.00	0.0	0.0	0.0

- Display the shape of the dataset

```
#Display Shape of Dataset
df.shape
```

[6] ✓ 0.0s

... (16598, 12)

- Display the column names

```
#Display Column Names
df.columns
```

[7] ✓ 0.0s

... Index(['Rank', 'Name', 'Platform', 'Year', 'Genre', 'Publisher', 'NA_Sales', 'EU_Sales', 'JP_Sales', 'Other_Sales', 'Global_Sales', 'Region'], dtype='str')

- Display the dataset information

```
#Display Dataset Information
df.info()
```

[8] ✓ 0.0s

... <class 'pandas.DataFrame'>
RangeIndex: 16598 entries, 0 to 16597
Data columns (total 12 columns):
Column Non-Null Count Dtype

0 Rank 16598 non-null int64
1 Name 16598 non-null str
2 Platform 16598 non-null str
3 Year 16327 non-null float64
4 Genre 16598 non-null str
5 Publisher 16540 non-null str
6 NA_Sales 16598 non-null float64
7 EU_Sales 16598 non-null float64
8 JP_Sales 16598 non-null float64
9 Other_Sales 16598 non-null float64
10 Global_Sales 16598 non-null float64
11 Region 335 non-null str
dtypes: float64(6), int64(1), str(5)
memory usage: 1.5 MB

- Display the datatypes

```
#Display Data Types
df.dtypes
```

```
Rank      int64
Name      str
Platform  str
Year      float64
Genre     str
Publisher  str
NA_Sales  float64
EU_Sales  float64
JP_Sales  float64
Other_Sales float64
Global_Sales float64
Region    str
dtype: object
```

- Check the missing values

```
#Check Missing Values
df.isnull().sum()
```

```
Rank      0
Name      0
Platform  0
Year      271
Genre     0
Publisher  58
NA_Sales  0
EU_Sales  0
JP_Sales  0
Other_Sales 0
Global_Sales 0
Region    16263
dtype: int64
```

- Check the duplicate records

```
#Check Duplicate Records
df.duplicated().sum()
```

```
np.int64(0)
```

- Display the descriptive statistics

```
#Display Descriptive Statistics
df.describe()
```

	Rank	Year	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sales
count	16598.000000	16327.000000	16598.000000	16598.000000	16598.000000	16598.000000	16598.000000
mean	8300.605254	2006.406443	0.264667	0.146652	0.077782	0.048063	0.537441
std	4791.853933	5.828981	0.816683	0.505351	0.309291	0.188588	1.555028
min	1.000000	1980.000000	0.000000	0.000000	0.000000	0.000000	0.010000
25%	4151.250000	2003.000000	0.000000	0.000000	0.000000	0.000000	0.060000
50%	8300.500000	2007.000000	0.080000	0.020000	0.000000	0.010000	0.170000
75%	12449.750000	2010.000000	0.240000	0.110000	0.040000	0.040000	0.470000
max	16600.000000	2020.000000	41.490000	29.020000	10.220000	10.570000	82.740000

- Count all the Unique values

```

> ✓ 0.0s
#Count Unique Values
df.nunique()

Rank      16598
Name      11493
Platform    31
Year       39
Genre      12
Publisher   578
NA_Sales   409
EU_Sales   305
JP_Sales   244
Other_Sales 157
Global_Sales 623
Region      3
dtype: int64

```

- Display all the unique values of the particular columns

```

> ✓ 0.0s
#Display Unique Values of a Particular Column
df["Genre"].unique()

<StringArray>
[      'Sports',      'Platform',      'Racing', 'Role-Playing',
      'Puzzle',      'Misc',      'Shooter',  'Simulation',
      'Action',      'Fighting',      'Adventure',  'Strategy']
Length: 12, dtype: str

```

- Count the product categories

```

> ✓ 0.0s
#Count Product Categories
df["Genre"].value_counts()

Genre
Action      3316
Sports      2346
Misc        1739
Role-Playing 1488
Shooter     1310
Adventure   1286
Racing      1249
Platform     886
Simulation   867
Fighting     848
Strategy     681
Puzzle       582
Name: count, dtype: int64

```

- Count the Name (database preferred)

```
#Count Name
df["Name"].value_counts()
✓ 0.0s
```

Name	
Need for Speed: Most Wanted	12
FIFA 14	9
Madden NFL 07	9
LEGO Marvel Super Heroes	9
Ratatouille	9
...	..
Chou Ezaru wa Akai Hana: Koi wa Tsuki ni Shirube Kareru	1
Eiyuu Densetsu: Sora no Kiseki Material Collection Portable	1
Plushees	1
Woody Woodpecker in Crazy Castle 5	1
Know How 2	1

Name: count, Length: 11493, dtype: int64

- Show the value counts

```
df["Publisher"].value_counts()
✓ 0.0s
```

Publisher	
Electronic Arts	1351
Activision	975
Namco Bandai Games	932
Ubisoft	921
Konami Digital Entertainment	832
...	...
Inti Creates	1
Takuyo	1
Interchannel-Holon	1
Rain Games	1
UIG Entertainment	1

Name: count, Length: 578, dtype: int64

- Calculate the Japan Sales

```
#Calculate Japan Sales Revenue
df["JP_Sales"].mean()
✓ 0.0s
```

np.float64(0.07778166044101699)

- Find Global and Europe Sales

```
#Find Global Sales
df["Global_Sales"].max()
[23] ✓ 0.0s
```

... np.float64(82.74)

```
#Find Europe Sales
df["EU_Sales"].min()
[24] ✓ 0.0s
```

... np.float64(0.0)

- The Correlation Matrix

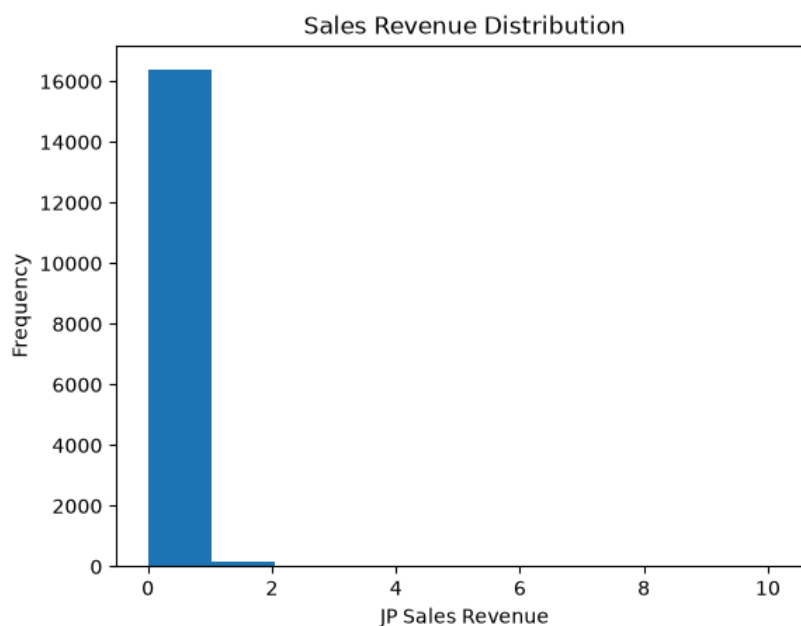
```
#Correlation Matrix
df.corr(numeric_only=True)
```

	Rank	Year	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sales
Rank	1.000000	0.178814	-0.401362	-0.379123	-0.267785	-0.332986	-0.427407
Year	0.178814	1.000000	-0.091402	0.006014	-0.169316	0.041058	-0.074735
NA_Sales	-0.401362	-0.091402	1.000000	0.767727	0.449787	0.634737	0.941047
EU_Sales	-0.379123	0.006014	0.767727	1.000000	0.435584	0.726385	0.902836
JP_Sales	-0.267785	-0.169316	0.449787	0.435584	1.000000	0.290186	0.611816
Other_Sales	-0.332986	0.041058	0.634737	0.726385	0.290186	1.000000	0.748331
Global_Sales	-0.427407	-0.074735	0.941047	0.902836	0.611816	0.748331	1.000000

- Plot the Histogram

```
#Histogram
plt.hist(df["JP_Sales"])
plt.title("Sales Revenue Distribution")
plt.xlabel("JP Sales Revenue")
plt.ylabel("Frequency")
plt.show()
```

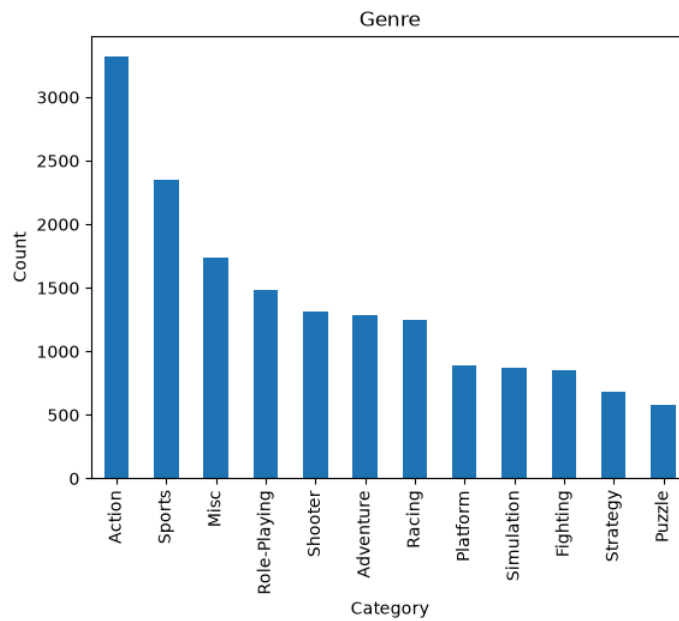
- Histogram of Japan Sales



- Plot the Bar Graph

```
#Bar chart
df["Genre"].value_counts().plot(kind="bar")
plt.title("Genre")
plt.xlabel("Category")
plt.ylabel("Count")
plt.show()
```

- Bar Graph of the Genre Category

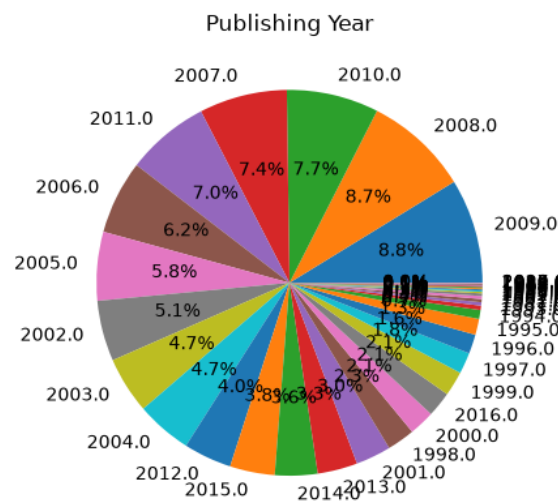


- Create the Pie Chart of the data

```
#Pie chart
df["Year"].value_counts().plot(kind="pie", autopct="%1.1f%%")
plt.title("Publishing Year")
plt.ylabel("")
plt.show()
```

✓ 0.1s

- Pie Chart of the Publishing Year

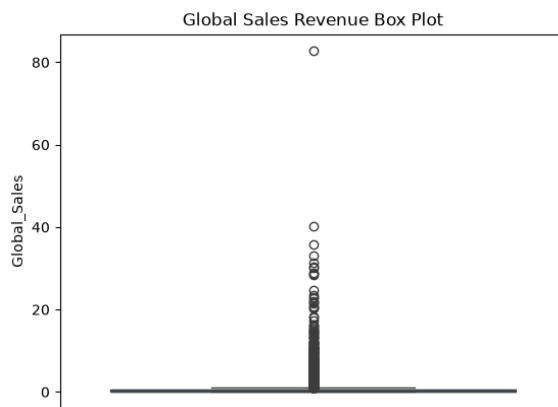


- Plot the Box Plot

```
#Box plot
sns.boxplot(y=df["Global_Sales"])
plt.title("Global Sales Revenue Box Plot")
plt.show()
```

✓ 0.0s

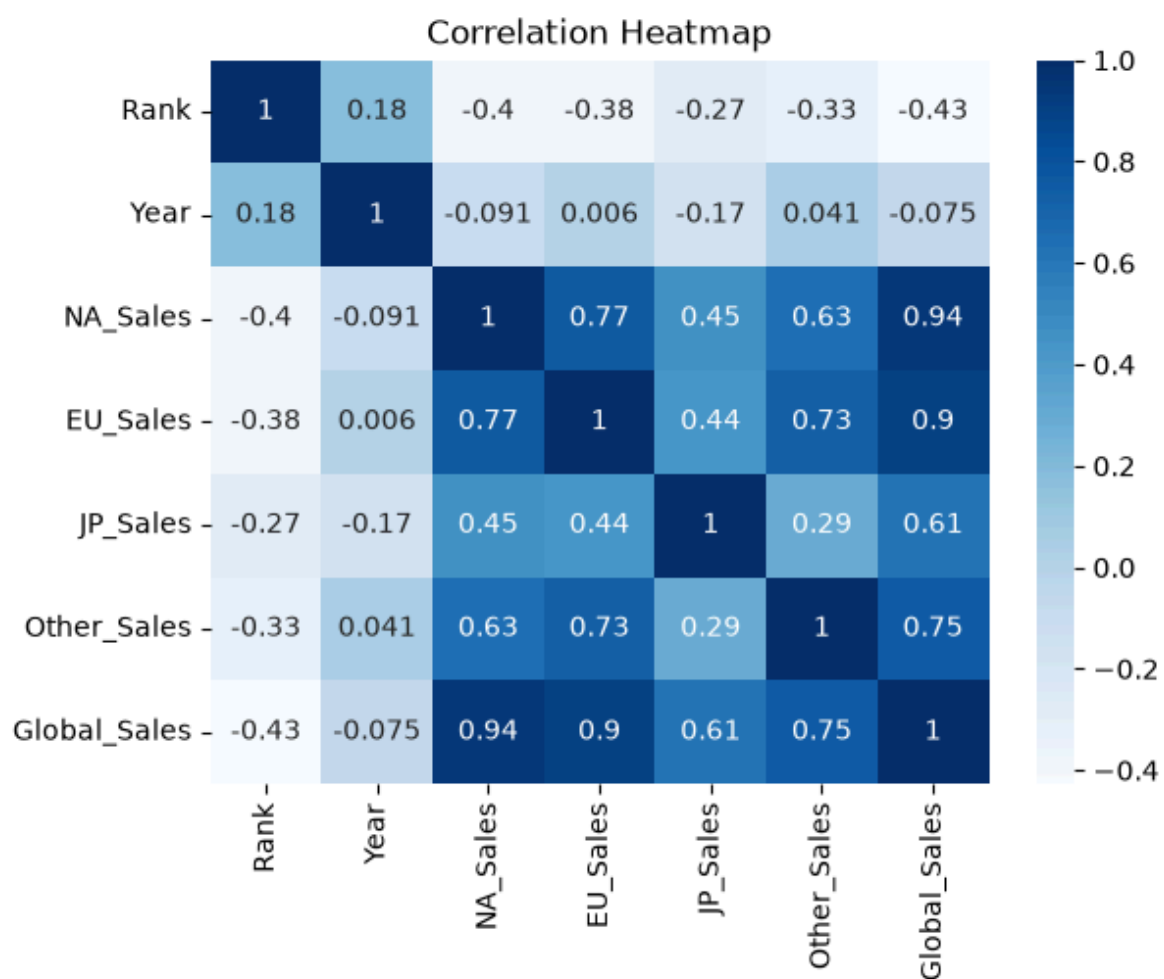
- Box Plot of Global Sales



- Create the Correlation Heatmap

```
#Correlation Heatmap
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="Blues")
plt.title("Correlation Heatmap")
plt.show()
```

- Heatmap of the dataset



B) Analyze customer data to identify trends and patterns that can be used for business decision-making.

- Using the same dataset or different, Print the Genre

```
print(df["Genre"].value_counts())
```

✓ 0.0s

Genre	
Action	3316
Sports	2346
Misc	1739
Role-Playing	1488
Shooter	1310
Adventure	1286
Racing	1249
Platform	886
Simulation	867
Fighting	848
Strategy	681
Puzzle	582

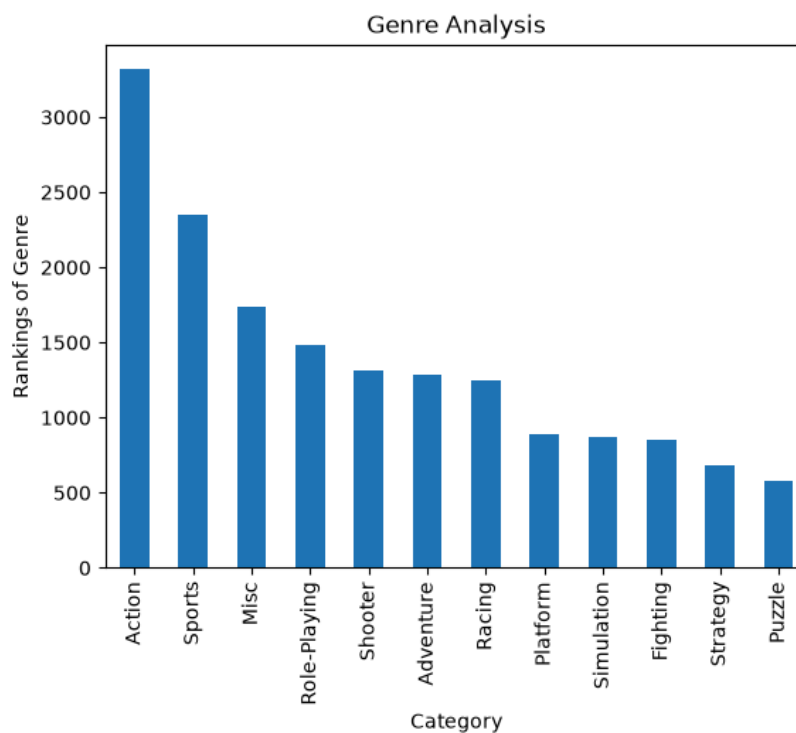
Name: count, dtype: int64

- Plot the Analysis table

```
df["Genre"].value_counts().plot(kind="bar")  
plt.title("Genre Analysis")  
plt.xlabel("Category")  
plt.ylabel("Rankings of Genre")  
plt.show()
```

✓ 0.1s

- The Genre Analysis table



- Print the Name

```
print(df["Name"].value_counts())
```

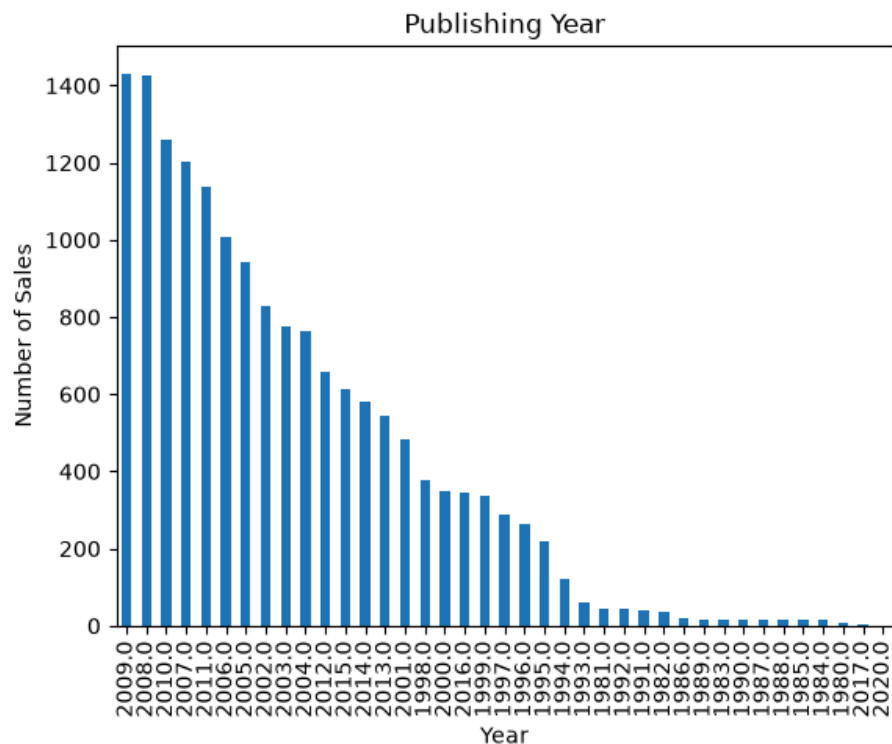
Name	Count
Need for Speed: Most Wanted	12
FIFA 14	9
Madden NFL 07	9
LEGO Marvel Super Heroes	9
Ratatouille	9
Chou Ezarū wa Akai Hana: Koi wa Tsuki ni Shirube Kareru	1
Eiyū Densetsu: Sora no Kiseki Material Collection Portable	1
Plushees	1
Woody Woodpecker in Crazy Castle 5	1
Know How 2	1

Name: count, Length: 11493, dtype: int64

- Create bar plot of the publishing year

```
df["Year"].value_counts().plot(kind="bar")
plt.title("Publishing Year")
plt.xlabel("Year")
plt.ylabel("Number of Sales")
plt.show()
```

- The plot of the publishing year



- Print the Japan Sales

```
print(df["JP_Sales"].value_counts())
```

✓ 0.0s

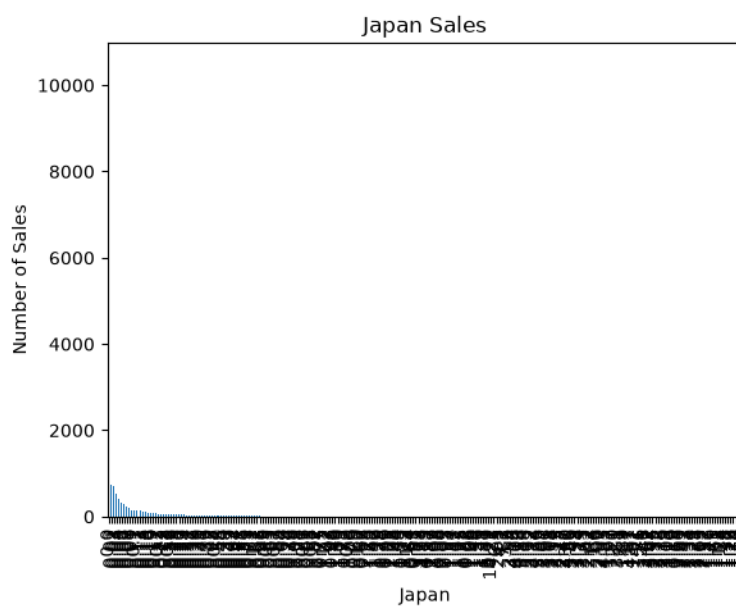
```
JP_Sales
0.00    10455
0.02     728
0.01     704
0.03     528
0.04     397
...
1.26      1
1.28      1
1.14      1
1.17      1
1.02      1
Name: count, Length: 244, dtype: int64
```

- Plot a graph for the Japan Sales

```
df["JP_Sales"].value_counts().plot(kind="bar")
plt.title("Japan Sales")
plt.xlabel("Japan")
plt.ylabel("Number of Sales")
plt.show()
```

✓ 0.6s

- The plot of the Japan Sale



- In the same way do with the Europe Sales

```
print(df["EU_Sales"].value_counts())
```

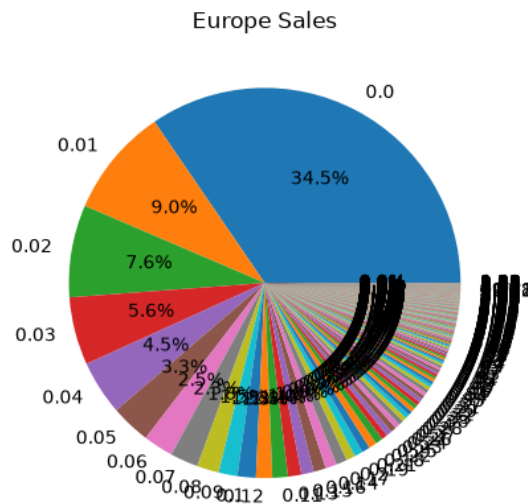
✓ 0.0s

```
EU_Sales
0.00    5730
0.01    1496
0.02    1269
0.03     934
0.04     748
...
2.11      1
1.85      1
2.43      1
1.49      1
1.23      1
Name: count, Length: 305, dtype: int64
```

- Create Pie chart for the Europe Sales

```
df["EU_Sales"].value_counts().plot(kind="pie", autopct="%1.1f%%")
plt.title("Europe Sales")
plt.ylabel("")
plt.show()
```

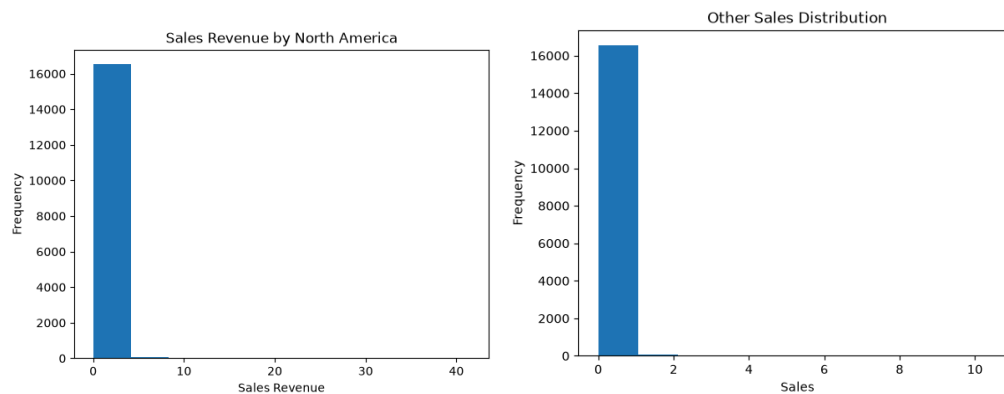
- Show the Pie Chart



- Create a chart for North America and Other Sales

```
plt.hist(df["NA_Sales"], bins=10)
plt.title("Sales Revenue by North America")
plt.xlabel("Sales Revenue")
plt.ylabel("Frequency")
plt.show()
```

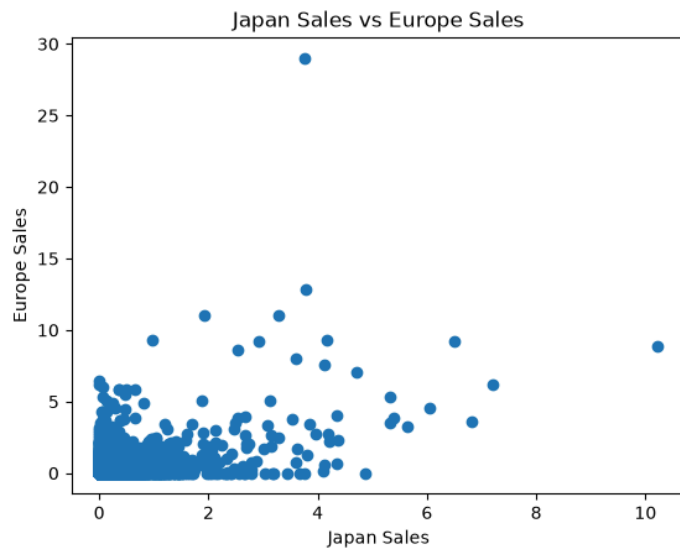
- Both will be done in the same way



- Now create the Scatter plot between two sales, for eg : Japan Sales vs Europe Sales

```
plt.scatter(df["JP_Sales"], df["EU_Sales"])
plt.title("Japan Sales vs Europe Sales")
plt.xlabel("Japan Sales")
plt.ylabel("Europe Sales")
plt.show()
```

- The Scatter Plot



- Create a Histogram for the correlation

```
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="Oranges")
plt.title("Correlation Heatmap")
plt.show()
```

- Histogram

